



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

Subject: Computer Vision Laboratory

Experiment - 7

Aim: To apply mean-shift clustering on image features for unsupervised segmentation.

Software Tools Required: VS Code, Python 3.12

Theory:

1. Introduction

Image segmentation is about dividing an image into different meaningful parts, like separating objects from the background. When doing this unsupervised, we don't rely on pre-labeled data. Instead, we look for natural groupings in the data — for example, pixels that look or behave similarly. Mean-shift clustering is a widely used unsupervised method for this. It groups pixels based on the density of data points in a chosen feature space, without needing to know beforehand how many groups there are.

2. What is Mean-Shift?

Mean-shift is a clustering method that works by finding areas in a dataset where a lot of points are close together — these areas are called modes or density peaks. The idea is simple:

- Imagine dropping a window (a circular or spherical region) somewhere in the data.
- Look at all the points inside that window and move the window toward the average (the “mean”) position of those points.
- Repeat this until the window stops moving, meaning it has found a dense region.
- Points that end up in the same dense region are considered part of the same cluster.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

3. Using Mean-Shift on Images

To use mean-shift for segmenting an image, we first need to decide how to describe each pixel. We usually combine two kinds of information:

1. Color information – how the pixel looks (e.g., RGB or Lab color values).

Constituent of Symbiosis International (Deemed University), Pune

2. Spatial information – where the pixel is located (its x, y coordinates in the image).

Each pixel becomes a point in this combined “feature space.” Mean-shift then groups together pixels that are similar in both color and position. This way, we get segments that represent meaningful regions in the image.

4. The Process Step-by-Step

1. Represent each pixel with a set of features (color + location).
2. Place a search window (a small region) over a pixel in the feature space.
3. Find all other pixels within that window.
4. Compute the average feature position of those pixels.
5. Shift the window to that average position.
6. Repeat steps 3–5 until the window stops moving (it has found a dense cluster).
7. Assign all pixels that end up in the same cluster to the same segment in the image.

5. Advantages

- No need to know how many segments (clusters) you want ahead of time. □
Can find irregularly shaped regions, not just circular or rectangular ones.
- Works well even if some data points (pixels) are outliers.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

7. Applications

- Detecting and separating objects in photos and videos.
- Medical imaging, for separating different tissues or abnormalities.
- Preprocessing for object recognition or scene understanding in computer vision systems.

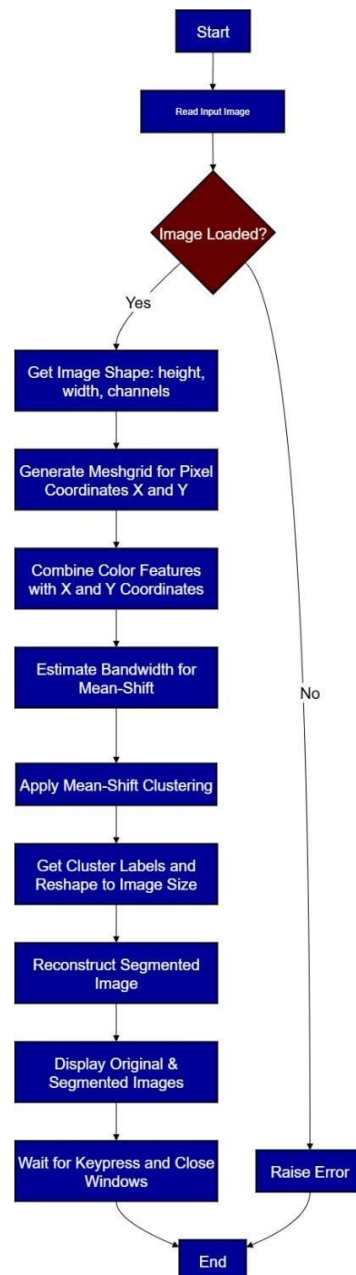
Constituent of Symbiosis International (Deemed University), Pune

Flowchart Part-

1:



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE



Constituent of Symbiosis International (Deemed University), Pune

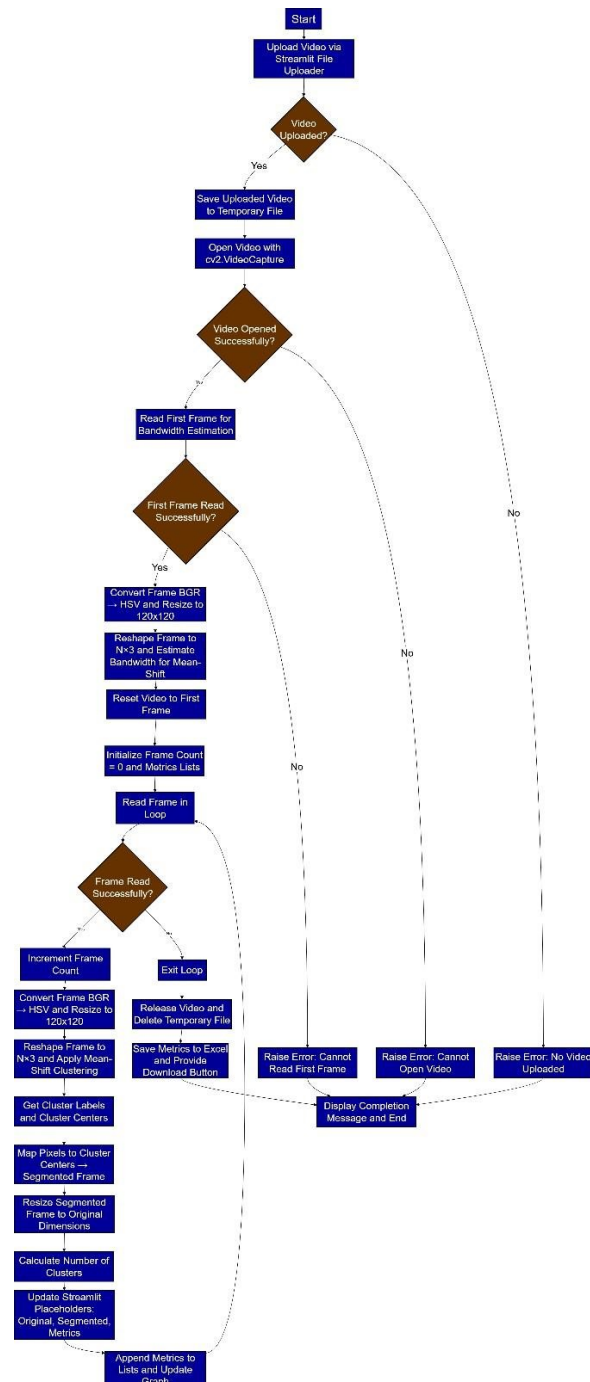
Part-2:





SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Part-3





SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

Program with Output

Program:

Part-1



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

```
import cv2
import numpy as np
from sklearn.cluster import MeanShift, estimate_bandwidth
import matplotlib.pyplot as plt

# ----- Step 1: Load Image -----
# Make sure "building.jpg" is in the same folder OR give full path
img = cv2.imread('building.jpg')
if img is None:
    raise FileNotFoundError("Image not found. Place 'building.jpg' in the project folder.")

# Convert BGR (OpenCV default) to RGB (for matplotlib display)
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
h, w, c = img.shape

# ----- Step 2: Create Feature Space [R,G,B,X,Y] -----
X, Y = np.meshgrid(np.arange(w), np.arange(h))
features = np.concatenate([
    img.reshape((-1, 3)), # Color features (R,G,B)
    X.reshape(-1, 1),    # X-coordinate
    Y.reshape(-1, 1)     # Y-coordinate
], axis=1)

# ----- Step 3: Estimate bandwidth and apply Mean-Shift -----
bandwidth = estimate_bandwidth(features, quantile=0.1, n_samples=500)
print(f"Estimated Bandwidth = {bandwidth:.2f}")

ms = MeanShift(bandwidth=bandwidth, bin_seeding=True)
ms.fit(features)

labels = ms.labels_
cluster_centers = ms.cluster_centers_

# ----- Step 4: Reconstruct Segmented Image -----
segmented_img = cluster_centers[labels][:, :3].reshape(h, w, 3).astype(np.uint8)

# ----- Step 4.5: Get Number of Clusters -----
num_clusters = len(np.unique(labels))
print(f"Number of clusters found: {num_clusters}")

# ----- Step 5: Show Results -----
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(img)
plt.axis("off")
```

Constituent of Symbiosis International (Deemed University), Pune



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

```
# ----- Step 4: Reconstruct Segmented Image -----
segmented_img = cluster_centers[labels][:, :3].reshape(h, w, 3).astype(np.uint8)

# ----- Step 4.5: Get Number of Clusters -----
num_clusters = len(np.unique(labels))
print(f"Number of clusters found: {num_clusters}")

# ----- Step 5: Show Results -----
plt.figure(figsize=(12, 6))

plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(img)
plt.axis("off")

plt.subplot(1, 2, 2)
plt.title(f"Mean-Shift Segmentation\nClusters: {num_clusters}")
plt.imshow(segmented_img)
plt.axis("off")

plt.tight_layout()
plt.show()
```

Part-2

```
import cv2
import numpy as np
from sklearn.cluster import MeanShift, estimate_bandwidth

# ----- Load video -----
cap = cv2.VideoCapture("Walk.mp4") # change to 0 for webcam

if not cap.isOpened():
    raise FileNotFoundError("Video not found or unable to open.")

# ----- Setup Video Writer -----
fourcc = cv2.VideoWriter_fourcc(*'mp4v')
fps = int(cap.get(cv2.CAP_PROP_FPS)) or 20
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
out = cv2.VideoWriter("Walk_segmented.mp4", fourcc, fps, (width, height))

# ----- Estimate bandwidth once using first frame -----
ret, frame = cap.read()
if not ret:
    raise RuntimeError("Cannot read first frame for bandwidth estimation.")

frame_small = cv2.resize(frame, (120, 120))
features = frame_small.reshape(-1, 3) # color features only
bandwidth = estimate_bandwidth(features, quantile=0.2, n_samples=500)
if bandwidth is None or bandwidth <= 0:
```

Constituent of Symbiosis International (Deemed University), Pune



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

```
if bandwidth is None or bandwidth <= 0:
    bandwidth = 20

print(f"Estimated bandwidth: {bandwidth:.2f}")

# Reset video to first frame
cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
frame_count = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame_count += 1
    # Resize for faster processing
    frame_small = cv2.resize(frame, (120, 120))
    features = frame_small.reshape(-1, 3)

    # Apply Mean Shift
    ms = MeanShift(bandwidth=bandwidth, bin_seeding=True)
    ms.fit(features)
    labels = ms.labels_
    cluster_centers = np.uint8(ms.cluster_centers_)

    # Map pixels to cluster centers
    segmented = cluster_centers[labels].reshape(frame_small.shape)

    # Resize back to original size
    segmented_big = cv2.resize(segmented, (frame.shape[1], frame.shape[0]))

    # Calculate number of clusters
    num_clusters = len(np.unique(labels))
    print(f"Frame {frame_count}: Clusters = {num_clusters}")

    # Display results
    cv2.putText(segmented_big, f"Bandwidth: {bandwidth:.2f}", (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2)
    cv2.putText(segmented_big, f"Clusters: {num_clusters}", (10, 60),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2)

    cv2.imshow("Original Video", frame)
    cv2.imshow("Mean-Shift Segmentation", segmented_big)

    # Save segmented frame
    out.write(segmented_big)

    # Quit on 'q'
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

Constituent of Symbiosis International (Deemed University), Pune



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

```
# Calculate number of clusters
num_clusters = len(np.unique(labels))
print(f"Frame {frame_count}: Clusters = {num_clusters}")

# Display results
cv2.putText(segmented_big, f"Bandwidth: {bandwidth:.2f}", (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2)
cv2.putText(segmented_big, f"Clusters: {num_clusters}", (10, 60),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2)

cv2.imshow("Original Video", frame)
cv2.imshow("Mean-Shift Segmentation", segmented_big)

# Save segmented frame
out.write(segmented_big)

# Quit on 'q'
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
out.release()
cv2.destroyAllWindows()
print(f"Segmentation video saved as 'Walk_segmented.mp4' with {frame_count} frames processed.")
```

Part-3

```
import streamlit as st
import cv2
import numpy as np
from sklearn.cluster import MeanShift, estimate_bandwidth
import time
import pandas as pd
import matplotlib.pyplot as plt
import os

# --- Streamlit App Setup ---
st.set_page_config(
    page_title="Mean-Shift Video Segmentation",
    layout="wide",
    initial_sidebar_state="expanded"
)

st.title("☞ Mean-Shift Video Segmentation")
st.markdown("Upload a video to see real-time color segmentation using the Mean-Shift algorithm.")
st.markdown("---")

# --- User Inputs ---
with st.sidebar:
    st.header("Settings")
    uploaded_file = st.file_uploader("Choose a video file", type=['mp4', 'mov', 'avi'])
    resize_dimensions = st.slider(
```

```
        "Resize Width for Processing (for speed)",
        min_value=60, max_value=320, value=120, step=20
    )

    st.info("The algorithm processes a smaller video for speed, then scales the result back up.")
    run_button = st.button("Start Segmentation")

# --- Main Logic ---
if uploaded_file is not None and run_button:
    # Save uploaded video to a safe temp file
    video_path = "uploaded_video.mp4"
    with open(video_path, "wb") as f:
        f.write(uploaded_file.read())

    # --- Video Loading ---
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        st.error("Error: Unable to open video file.")
        st.stop()

    fps = cap.get(cv2.CAP_PROP_FPS) or 20
    original_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    original_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

    st.subheader("Processing Video...")
    st.text(f"Original Dimensions: {original_width}x{original_height}, FPS: {fps:.2f}")
```



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

```
# --- Bandwidth Estimation ---
with st.spinner("Estimating bandwidth from the first frame..."):
    ret, frame = cap.read()
    if not ret:
        st.error("Cannot read the first frame.")
        st.stop()

    frame_hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    frame_small = cv2.resize(frame_hsv, (resize_dimensions, resize_dimensions))
    features = frame_small.reshape(-1, 3)
    bandwidth = estimate_bandwidth(features, quantile=0.2, n_samples=1000)
    if bandwidth is None or bandwidth <= 0:
        bandwidth = 20
    st.success(f"Estimated Bandwidth: {bandwidth:.2f}")

# Reset video to first frame
cap.set(cv2.CAP_PROP_POS_FRAMES, 0)

# --- Display Placeholders ---
col1, col2 = st.columns(2)
with col1:
    st.subheader("Original Video")
    original_video_placeholder = st.empty()
with col2:
    st.subheader("Mean-Shift Segmentation")
```

```
segmented_video_placeholder = st.empty()
metrics_placeholder = st.empty()

# --- Graph Placeholder ---
st.subheader("Clusters & Bandwidth vs Frames")
graph_placeholder = st.empty()

# --- Initialize lists to store data ---
frame_numbers = []
cluster_counts = []
bandwidths = []

frame_count = 0
start_total_time = time.time()

# --- Main Processing Loop ---
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    frame_count += 1
    start_frame_time = time.time()
```



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

```
# --- Process frame ---
frame_hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
frame_small = cv2.resize(frame_hsv, (resize_dimensions, resize_dimensions))
features = frame_small.reshape(-1, 3)

ms = MeanShift(bandwidth=bandwidth, bin_seeding=True)
ms.fit(features)
labels = ms.labels_
cluster_centers_hsv = np.uint8(ms.cluster_centers_)
segmented_hsv = cluster_centers_hsv[labels].reshape(frame_small.shape)
segmented_bgr = cv2.cvtColor(segmented_hsv, cv2.COLOR_HSV2BGR)
segmented_big = cv2.resize(segmented_bgr, (original_width, original_height))

end_frame_time = time.time()
num_clusters = len(np.unique(labels))

# --- Update video placeholders ---
original_video_placeholder.image(frame, channels="BGR", width='stretch')
segmented_video_placeholder.image(segmented_big, channels="BGR", width='stretch')
metrics_placeholder.markdown(f"""
    Frame {frame_count} | Clusters: {num_clusters} | 
    Processing Time: {end_frame_time - start_frame_time:.4f}s
""")
```

```
# --- Save data ---
frame_numbers.append(frame_count)
cluster_counts.append(num_clusters)
bandwidths.append(bandwidth)

# --- Update Matplotlib graph ---
fig, ax1 = plt.subplots(figsize=(10, 5))

# Dark theme background
fig.patch.set_facecolor('black')
ax1.set_facecolor('black')

# First axis (Clusters)
color_clusters = 'cyan'
ax1.set_xlabel('Frame Number', color='white')
ax1.set_ylabel('Number of Clusters', color=color_clusters)
ax1.plot(frame_numbers, cluster_counts, color=color_clusters, marker='o')
ax1.tick_params(axis='y', labelcolor=color_clusters)
ax1.tick_params(axis='x', color='white')

# Second axis (Bandwidth)
ax2 = ax1.twinx()
color_bandwidth = 'magenta'
ax2.set_ylabel('Bandwidth', color=color_bandwidth)
ax2.plot(frame_numbers, bandwidths, color=color_bandwidth)
```

```
# Second axis (Bandwidth)
ax2 = ax1.twinx()
color_bandwidth = 'magenta'
ax2.set_ylabel('Bandwidth', color=color_bandwidth)
ax2.plot(frame_numbers, bandwidths, color=color_bandwidth)
ax2.tick_params(axis='y', labelcolor=color_bandwidth)

# Title
plt.title('Mean-Shift Clustering Metrics Over Time', color='white')

graph_placeholder.pyplot(fig)
plt.close(fig) # free memory

# --- Cleanup ---
cap.release()
if os.path.exists(video_path):
    os.remove(video_path)

# --- Save Excel ---
df = pd.DataFrame({
    "Frame": frame_numbers,
    "Clusters": cluster_counts,
    "Bandwidth": bandwidths
})
excel_path = "MeanShift_Frame_Data.xlsx"
```

Constituent of Symbiosis International (Deemed University), Pune



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

```
df = pd.DataFrame({
    "Frame": frame_numbers,
    "Clusters": cluster_counts,
    "Bandwidth": bandwidths
})
excel_path = "MeanShift_Frame_Data.xlsx"
df.to_excel(excel_path, index=False)

st.balloons()
st.success("Video segmentation complete!")
st.markdown("---")
st.info(f"Total frames processed: {frame_count}")
st.info(f"Total time taken: {time.time() - start_total_time:.2f} seconds")

st.download_button(
    "📄 Download Excel Data",
    data=open(excel_path, "rb").read(),
    file_name="MeanShift_Frame_Data.xlsx",
    mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
)
```

Output:

Part-1

```
Estimated Bandwidth = 213.33
Number of clusters found: 4
```

Original Image



Mean-Shift Segmentation
Clusters: 4



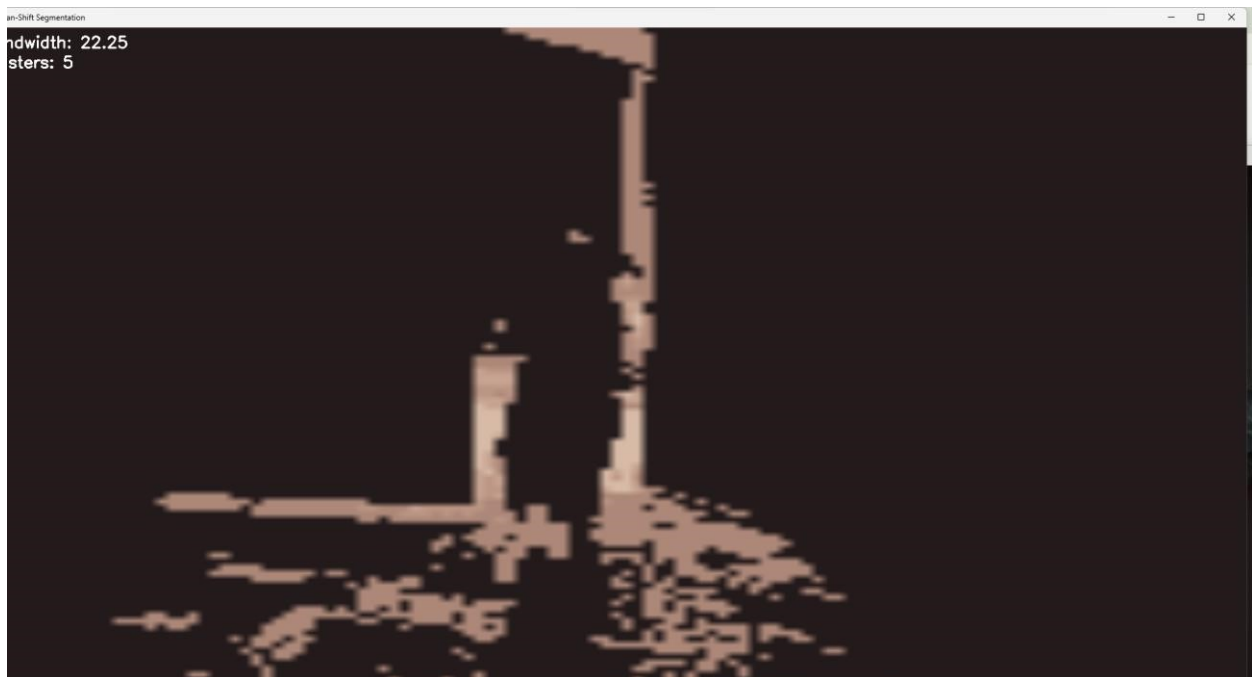
Constituent of Symbiosis International (Deemed University), Pune
Part-2



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

```
Estimated bandwidth: 22.25
Frame 1: Clusters = 6
Frame 2: Clusters = 2
Frame 3: Clusters = 2
Frame 4: Clusters = 3
Frame 3: Clusters = 2
Frame 4: Clusters = 3
Frame 4: Clusters = 3
Frame 5: Clusters = 3
Frame 5: Clusters = 3
Frame 6: Clusters = 5
Frame 6: Clusters = 5
Frame 7: Clusters = 5
Segmentation video saved as 'Walk_segmented.mp4' with 7 frames processed.
```

Part-3



Constituent of Symbiosis International (Deemed University), Pune

Conclusion/Inferences:

In this project, Mean-Shift clustering was applied to segment video frames based on color features. By estimating the bandwidth from the first frame and using downscaled frames, the computational complexity was significantly reduced, allowing practical processing of multi-frame video sequences. The segmentation results successfully grouped pixels into clusters representing distinct color regions, and the number of clusters per frame provided insights into the visual complexity of each scene.



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Overall, this approach demonstrates that Mean-Shift is effective for **unsupervised segmentation** and can be applied to videos with moderate resolution, offering a clear visualization of different regions in each frame. While computationally intensive for high-resolution or real-time applications, optimizations like frame resizing, color-only features, and pre-estimated bandwidth make it feasible for offline video processing tasks.